

Axiomander Design: Pure Specification Admission and Translation Pipeline

Status

Proposed

Motivation

Axiomander currently supports contracts over executable Python code, but many useful specifications require auxiliary predicates and model functions:

- `is_sorted(xs)`
- `is_permutation(xs, ys)`
- `replace_model(s, old, new)`
- `contains(xs, x)`
- `count(xs, x)`
- parser and normalization predicates
- object shape predicates

Without a specification language, contracts quickly become inexpressible.

The challenge is that these predicates are themselves programs. They frequently contain:

- recursion
- loops
- quantification
- sequence traversal

To reason about them mechanically, Axiomander must:

1. determine whether they are pure
2. determine whether they terminate
3. translate them into logical definitions
4. expose them to SMT and Rocq

The goal is to achieve this without introducing a separate specification language or user-visible library.

Design Principle

There is only one source language:

Python

Users write ordinary Python functions.

When a function becomes reachable from a contract, Axiomander attempts to admit it as a specification function.

Specification status is inferred rather than declared.

Example:

```
def is_sorted(xs):
    i = 0

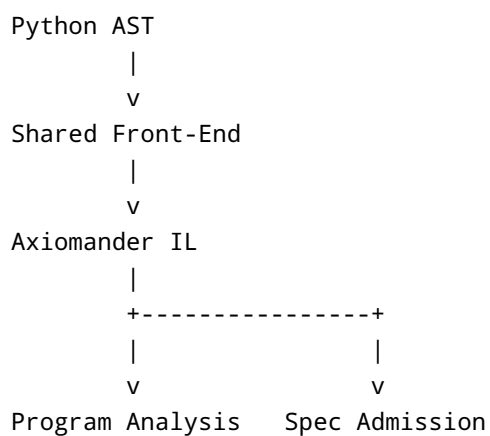
    while i + 1 < len(xs):
        if xs[i] > xs[i + 1]:
            return False
        i += 1

    return True

def insertion_sort(xs):
    ...
    assert post(is_sorted(result))
```

The use of `is_sorted` in a contract causes Axiomander to attempt specification admission.

High-Level Architecture




```
perform I/O
modify globals
modify nonlocals
perform heap mutation
perform database updates
call impure functions
perform reflection
use exec/eval
spawn tasks
perform synchronization
```

Allowed:

```
local variables
conditionals
loops
recursion
sequence access
string access
arithmetic
boolean logic
construction of immutable values
calls to admitted spec functions
```

2. Termination Gate

Specification functions must denote total logical functions.

Termination may be established by:

Structural recursion

```
def count(xs, x):
    if len(xs) == 0:
        return 0

    ...
    return count(xs[1:], x)
```

Loop variants

```
while i < len(xs):  
    ...  
    i += 1
```

Variant:

```
len(xs) - i
```

Trusted admission

When termination cannot be established mechanically.

3. Translation Gate

The admitted subset must be translatable into Pure IR.

Unsupported constructs are rejected.

4. Dependency Gate

All called functions must be either:

```
admitted spec functions  
trusted spec functions  
logical primitives
```

Pure IR

Pure IR is a logical subset of the Axiomander IL.

Expressions

```
Literal  
Variable  
IfExpr
```

```
Call
UnaryOp
BinaryOp
Compare
Len
Index
Slice
Tuple
List
Quantifier
```

Definitions

```
PureDef
  name
  params
  body
  decreases
```

Loop Handling

Pure IR contains no imperative loops.

Loops are normalized before translation.

Example:

```
def is_sorted(xs):
    i = 0

    while i + 1 < len(xs):
        if xs[i] > xs[i + 1]:
            return False

        i += 1

    return True
```

becomes:

```
fix go(i):  
  if i + 1 < len(xs):  
    xs[i] <= xs[i+1] and go(i+1)  
  else:  
    True
```

or alternatively:

```
forall i.  
  0 <= i < len(xs)-1  
  ->  
  xs[i] <= xs[i+1]
```

The recursive form should be implemented first because it is easier to derive mechanically.

Recursive Handling

Recursive specifications are translated directly into logical recursive definitions.

Example:

```
def count(xs, x):  
  if len(xs) == 0:  
    return 0  
  
  return (  
    (1 if xs[0] == x else 0)  
    + count(xs[1:], x)  
  )
```

becomes:

```
Fixpoint count(xs, x)
```

in Rocq.

Trusted Specification Functions

Not all useful specifications can be admitted automatically.

Axiomander therefore supports trusted specifications.

Example:

```
@trusted_spec(  
    reason="External model of Python string replacement"  
)  
def replace_model(s, old, new):  
    ...
```

Trusted specifications:

- bypass purity checking
- bypass termination checking
- may participate in proofs
- generate proof debt

Proof Debt

Every trusted item is recorded.

Example report:

```
Proof Summary  
  
Proved Obligations: 182  
  
Trusted Specifications:  
    replace_model  
    parse_json_model  
  
Assumptions:  
    3  
  
Unresolved:  
    0
```

Trust must never be invisible.

Specification Status

Each candidate receives exactly one status.

ADMITTED_TRANSPARENT

Definition available to:

- SMT
- Rocq
- unfolding
- symbolic execution

Example:

```
is_sorted  
count  
contains
```

ADMITTED_OPAQUE

Definition hidden.

Only contract summaries are exposed.

Useful for scaling.

TRUSTED

Accepted without proof.

Produces proof debt.

REJECTED

Cannot be used in specifications.

Example:

```
def is_sorted(xs):  
    print(xs)  
    ...
```

Reason:

I/O detected

Backend Generation

Pure IR is translated to:

SMT

When definitions fit supported fragments.

Examples:

```
arithmetic  
arrays  
sequences  
simple recursion
```

Rocq

When richer reasoning is required.

Examples:

```
induction  
recursive predicates  
structural proofs
```

The Rocq backend should receive Pure IR rather than raw Python AST.

Caching

Specification hashes must include:

```
body
dependencies
trusted status
termination metadata
```

Changes invalidate dependent proofs.

Initial MVP

Implement:

1. Contract reachability analysis
2. Spec admission pass
3. Purity checking
4. Structural recursion checking
5. Simple loop variant inference
6. Pure IR
7. Rocq generation from Pure IR
8. Trusted specification support
9. Proof debt reporting

Defer:

- higher-order functions
 - generators
 - async
 - mutual recursion
 - user-defined well-founded relations
 - advanced quantifier synthesis
-

Long-Term Goal

Axiomander should allow users to write ordinary Python helper functions and automatically elevate them into logical definitions when used in contracts.

The user experience should be:

```
def is_sorted(xs):  
    ...  
  
assert post(is_sorted(result))
```

rather than introducing a separate specification language.

The verifier determines whether the helper function can be treated as logic, records any trust assumptions, and exposes an admitted logical definition to SMT and Rocq.